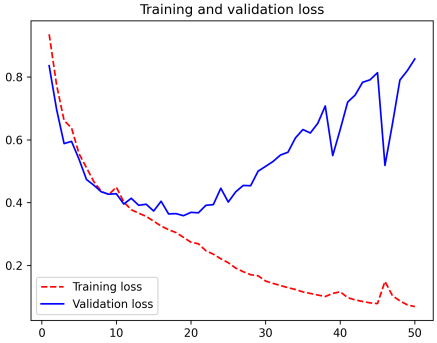
Biểu đồ Loss trong Huấn luyện

- Vì đây là phân loại 3 nhãn cho từng pixel, ta dùng Hàm mất mát: `sparse_categorical_crossentropy`.
- Mô hình hội tụ rất tốt ở giai đoạn đầu.
- Bắt đầu xuất hiện Overfitting sau epoch thứ 25.



Hình 11.5: Quá trình Training Loss

Biên dịch và Huấn luyện (Train)

```
model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=[foreground_iou],
)

callbacks = [
    keras.callbacks.ModelCheckpoint(
        "oxford_segmentation.keras", save_best_only=True
    )
]

history = model.fit(
    train_input_imgs, train_targets,
    epochs=50, batch_size=64,
    callbacks=callbacks,
    validation_data=(val_input_imgs, val_targets),
)
```

Kết quả Dự đoán (Prediction)

- Trực quan hóa kết quả dự đoán của mô hình tự thiết kế.
- Mô hình bắt được hình dáng của con mèo khá tốt (Khu vực Tiền cảnh).
- Tuy nhiên, các chi tiết nhỏ ở đường viền (như tai, râu) vẫn còn nhiễu và bị mờ (Artifacts).
- Để đạt độ chính xác SOTA, ta cần các mô hình lớn hơn rất nhiều.



Hình 11.6: Mặt nạ do mô hình dự đoán

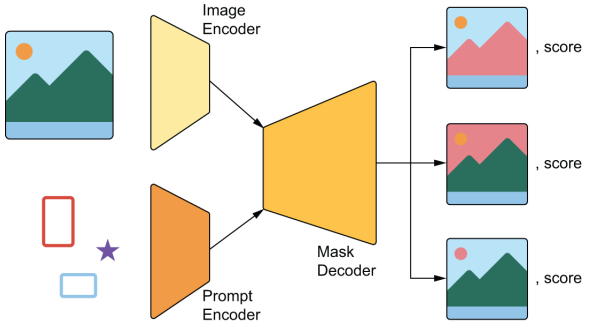
Kỷ nguyên của Foundation Models trong CV

- Natural Language Processing (NLP) đã thay đổi mãi mãi với sự xuất hiện của các Foundation Models (Như GPT-3, GPT-4) - Một mô hình huấn luyện một lần, giải quyết được mọi tác vụ (Zero-shot).
- Vào tháng 4/2023, Meta AI công bố **Segment Anything Model (SAM)**, đánh dấu cột mốc Foundation Model đầu tiên của ngành Thị giác Máy tính (Computer Vision).
- SAM có khả năng phân vùng "Bất cứ thứ gì" trong ảnh, mà không cần huấn luyện lại.

Kiến trúc của SAM

SAM gồm 3 thành phần cốt lõi:

1. **Image Encoder:** Vision Transformer (ViT) kích thước cực lớn, giải nén toàn bộ thông tin ảnh.
2. **Prompt Encoder:** Biểu diễn các Lệnh nhắc của người dùng (Điểm, Hộp, Text).
3. **Mask Decoder:** Một mạng nhẹ hợp nhất thông tin từ Image và Prompt để xuất ra Mặt nạ (Mask).



Hình 11.7: Kiến trúc tổng thể của Segment Anything

Siêu Dữ liệu SA-1B

- Sức mạnh của SAM đến từ Dữ liệu. Meta AI đã xây dựng tập dữ liệu **SA-1B (Segment Anything 1-Billion)**.
- Khối lượng khổng lồ: **11 triệu hình ảnh** với hơn **1.1 Tỷ mặt nạ phân vùng** chất lượng cao (gấp 400 lần các tập dữ liệu trước đó).
- Nhờ vậy, SAM có khả năng phân vùng Zero-shot với mọi thể loại vật thể từ sinh học, phương tiện, đến y tế.



Hình 11.8: Mật độ mặt nạ trong tập dữ liệu SA-1B

Khởi tạo SAM từ KerasHub

Mọi thứ đều có sẵn trong KerasHub, ta gọi mô hình siêu lớn với Preset `sam_huge_sa1b`.

```
import keras_hub

# Instantiates SAM and downloads its weights
model = keras_hub.models.ImageSegmenter.from_preset("sam_huge_sa1b")

print(model.count_params())
# Output: 641090864
```

Với 641 triệu tham số, đây là mô hình lớn nhất mà chúng ta sử dụng! Tốc độ tiến hóa của Pre-trained Models tỷ lệ thuận với lượng dữ liệu cung cấp (Sẽ học ở Chương 16).

Môi trường Thử nghiệm

- Ta sẽ thử nghiệm sức mạnh của SAM trên một bức ảnh thực tế chứa rất nhiều loại Trái cây (Chuối, Đào, Xoài).
- Bức ảnh này có cấu trúc phức tạp với vật thể chồng chéo và bóng đổ.
- Mục tiêu: Gọi API của SAM để phân vùng chúng thông qua các kiểu **Prompting (Lệnh nhắc)** khác nhau.



Hình 11.9: Ảnh Trái cây đầu vào

Tiền xử lý Ảnh cho SAM (Padding)

SAM yêu cầu ảnh đầu vào có kích thước cố định 1024×1024. Ta không được bóp méo tỷ lệ khung hình, nên phải dùng Padding.

```
from keras import ops

image_size = (1024, 1024)

def resize_and_pad(x):
    # Resizes the image to 1024 on its longest side,
    # then pads the remaining pixels with 0 to keep aspect ratio
    return ops.image.resize(x, image_size, pad_to_aspect_ratio=True)

image = resize_and_pad(image_array)
```

Hàm tiện ích (Utilities)

Chúng ta viết các hàm hiển thị ảnh với Matplotlib để vẽ Bounding Box và Points.

```python
import matplotlib.pyplot as plt
from keras import ops

def show_image(image, ax):
    ax.imshow(ops.convert_to_numpy(image).astype("uint8"))

def show_points(points, ax):
    x, y = points[:, 0], points[:, 1]
    ax.scatter(x, y, c="green", marker="*", s=375, ec="white", lw=1.25)

def show_box(box, ax):
    box = box.reshape(-1)
    x0, y0 = box[0], box[1]
    w, h = box[2] - box[0], box[3] - box[1]
    ax.add_patch(plt.Rectangle((x0, y0), w, h, ec="red", fc="none", lw=2))
```

Hàm Trích xuất Mặt nạ (Get Mask)

Mỗi lần được Prompt, SAM sẽ trả về 4 Mặt nạ ứng viên dự đoán tốt nhất, xếp hạng theo điểm chất lượng.

```
def get_mask(sam_outputs, index=0):
    # Select the mask candidate with highest score (index 0)
    mask = sam_outputs["masks"][0][index]
    mask = np.expand_dims(mask, axis=-1)
    # Resize mask back to fit the image
    mask = resize_and_pad(mask)
    # Convert logits into a boolean mask
    return ops.convert_to_numpy(mask) > 0.0
```

Hàm này sẽ trích xuất ra một ma trận Boolean (True/False) chỉ định các Pixel thuộc khu vực Tiền cảnh.

Mặt nạ thay thế (Alternative Masks)

- Khi Prompt một điểm nhập nhằng (Ví dụ: Giữa nải chuối).
- SAM sẽ nội suy và sinh ra 3 phương án dự phòng (Alternative Masks) với nhiều cấp độ bao phủ khác nhau.
- Điểm thú vị là: Một phương án sẽ bắt riêng lẻ 1 trái chuối, phương án khác bao trùm cả nải chuối! (Rất thông minh).



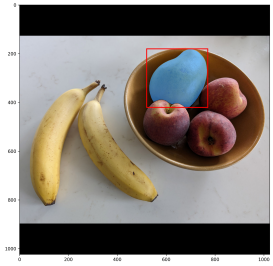
Hình 11.10: Các Mặt nạ thay thế của SAM

Nhắc bằng Hộp giới hạn (Box Prompting)

Cung cấp 1 hộp bao phủ trái xoài (Tọa độ Góc trên-trái và Góc dưới-phải).

```
input_box = np.array([
    [520, 180], # Top-left corner
    [770, 420], # Bottom-right corner
])

outputs = model.predict(
    {
        "images": ops.expand_dims(img,
    axis=0),
        "boxes": ops.expand_dims(input_box
    , axis=(0, 1)),
    }
)
```



Hình 11.11: SAM lập tức trả về Mặt nạ Xoài chính xác

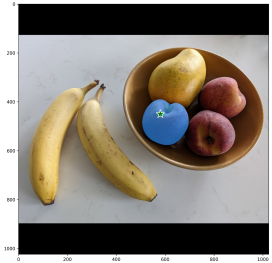
Nhắc bằng Điểm đơn (Single Point Prompting)

Nhập đúng 1 tọa độ rơi vào quả Đào. (Nhãn 1 nghĩa là Tiền cảnh, chọn cái này!)

```
# Coordinates of our point
input_point = np.array([[580, 450]])

# 1 means foreground, 0 means background
input_label = np.array([1])

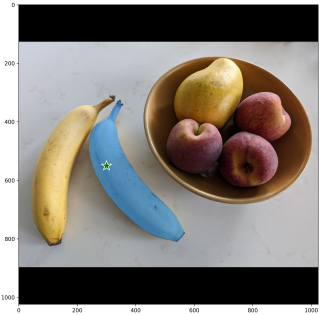
outputs = model.predict(
    {
        "images": ops.expand_dims(image,
    axis=0),
        "points": ops.expand_dims(
    input_point, axis=0),
        "labels": ops.expand_dims(
    input_label, axis=0),
    }
)
```



Hình 11.12: SAM lan truyền điểm đó thành Mặt nạ Đào

Sức mạnh nội suy của Point Prompt

- Tương tự, nếu ta truyền Tọa độ điểm của một quả Chuối bị khuất.
- Kể cả khi quả chuối bị che bởi bóng đổ hay trái cây khác, công cụ ViT khổng lồ của SAM vẫn nội suy (Interpolate) ra đường viền hoàn mỹ.
- Quá trình này diễn ra theo thời gian thực (Real-time) trong trình duyệt nhờ Mask Decoder siêu nhẹ.



Hình 11.13: Phân vùng Chuối bằng 1 Point Prompt